

5 V's of Big Data

<u>Volume</u> ↓ Data at Rest	<u>Velocity</u> ↓ Data in Motion	<u>Variety</u> ↓ Data in Many forms	<u>Veracity</u> ↓ Data in Doubt	<u>Value</u> ↓ Data into money.	
Another 5 →	Validity	Variability	Venue	Vocabulary	Vagueness

DIKW [Data Information Knowledge Wisdom]



Storage as [NAS (Network Attached Storage)] and [SAN (Storage Area Network)]

Data is moved to compute nodes at compute time.

↓
When we have to deal with huge data, getting back the data to the processor becomes the bottleneck.

↓
adding load to system → performance ↓, ↓, ↓

Soln is — HADOOP → commodity Hardware.

Core Principles:

1. Scale-Out rather than Scale-Up
2. Bring code to data rather than data to code
3. Deal with failure — they are common
4. Abstract complexity of distributed & concurrent application

Hadoop Daemons — five separate daemons.

1. Name Node

2. Secondary Name Node

3. Data Node

4. Job Tracker — Manages MapReduce jobs,

5. Task Tracker — Responsible for instantiating and monitoring individual Map & Reduce tasks.

Big Data

Midterm - 4 topics →
Hadoop
Spark
Rhadoop
Pig Latin.

Hadoop

- Hadoop is a popular open-source framework for processing large datasets in a distributed computing environment.
- It helps organizations process vast amounts of data (structured and unstructured) efficiently and cost-effectively.
- It works on the concept of distributed storage and distributed processing.

Key Components of Hadoop:

1. HDFS

This is the storage layer of Hadoop.

It stores large files across multiple machines, ensuring data redundancy, fault-tolerance, and scalability.

2. MapReduce

This is the processing layer of Hadoop.

It allows data processing to be divided into small tasks that can run in parallel across multiple machines.

HDFS (Hadoop Distributed file System)

- HDFS is the primary storage system of Hadoop.
- It is designed to store large files across many machines in a fault-tolerant manner.

Architecture :

1. Name Node: This is the master node that manages the metadata (information about where the data is stored) of the files in the system.
2. Data Nodes: These are the worker nodes where the actual data is stored. Each file is divided into blocks and distributed across these DataNodes.

Key features: Fault Tolerance, Scalability, Write once Read Many

- Block-Based Storage: files in HDFS are split into fixed-size blocks (typically 128 MB (or) 256 MB).

These blocks are distributed across multiple nodes in the cluster.

- Replication: Each block is replicated across multiple Datanodes (typically 3 times) to ensure fault-tolerance. If a node fails, data can still be retrieved from another replica.

- High Throughput: HDFS is optimized for reading large datasets rather than for low-latency random reads.

Name Node (Master Node)	Datanode (Worker Node)	Secondary Name Node
- Manage Metadata (file name, location, replications). - Does not store actual data. - Maintain a directory structure similar to traditional file system.	- Store Actual Data. - Respond to NameNode's request (read/write/replication)	- Acts as a backup for the NameNode. - Periodically merges NameNode's logs and snapshots.

Map Reduce

MapReduce is a programming model for processing large dataset in parallel across a distributed cluster of computer.

It has two main phases:

• Map Phase:

The input data is divided into smaller chunks. (usually one chunk per blocks in HDFS).

The Map function processes these chunks and outputs a set of Key-value pairs (intermediate results).

• Shuffle and Sort:

After the map phase, the system sorts and groups the intermediate Key-value pairs by Key. This process is called shuffling.

• Reduce Phase:

The Reduce function takes these sorted and grouped Key-value pairs and processes them to produce the final output (aggregated results).

Example of mapReduce Job: -

Input: Hello Hadoop Hadoop

Map o/p: (Hello, 1) (Hadoop, 1) (Hadoop, 1)

Reduce phase: (Hello, 1) (Hadoop, 2)

If you want to count the number of occurrences of each word in a large text file:

1. The MapReduce function would break the text into words and emit a Key-value pair for each word (eg- ("word", 1)).

2. The Reduce function would then sum the counts for each word to give the total occurrences.

Advantage of MapReduce:

1. Parallel Processing [Distributed tasks across multiple nodes]
2. Fault Tolerance [Automatic task re-execution on in case of failure]
3. Optimized for large Data Processing

YARN (Yet Another Resource Negotiator)

YARN is the resource management layer at Hadoop. It was introduced in Hadoop 2.0 to improve the scalability and resource utilization of the Hadoop ecosystem.

YARN is responsible for managing and scheduling resources in a multi-tenant cluster.

Components of YARN:

1. **Resource Manager (RM):** The master daemon that manages the resources and schedules jobs on the cluster.
2. **Node Manager (NM):** The worker node daemon that is responsible for managing resources and monitoring the health of the node.
3. **Application Master (AM):** For each application/job, an Application Master is created to negotiate resources with the RM and manage the execution of the application on the cluster.

Hadoop Ecosystem

Hadoop is not just about HDFS and MapReduce. There is a large ecosystem of projects that enhance its capabilities for various use cases!

1. Hive
2. Pig
3. HBase
4. Sqoop
5. Flume
6. Oozie
7. Zookeeper

1. Hive : A data warehouse system build on the top of the Hadoop, which allows SQL like queries (HiveQL) to be run on large dataset.
2. Pig : A platform for analyzing large datasets using a high-level scripting language called Pig Latin.
3. HBase : A No SQL database that runs on the top of HDFS, designed for real-time read/write access to large datasets.
4. Sqoop : A tool for transferring bulk data between Hadoop and relational databases.
5. Flume : A service for collecting, aggregating, and moving large amount of log data into HDFS.
6. Oozie : A workflow scheduler for Hadoop that allows users to manage and schedule complex data processing pipelines.
7. Zookeeper : A coordination service for distributed application ensuring synchronization and managing configuration.

for storage and Data Management — HBase
Hive
Pig

Data Processing — Spark
Tez

Data Ingestion — Sqoop
Flume

Workflow Management — Oozie

Search & Indexing — Solr/Lucene

Hadoop Cluster Setup

A Hadoop cluster consists of several machines (nodes) working together. There are two types of Nodes:

1. Master Node: This node coordinates the job scheduling and resource management (Resource Management, NameNode)
2. Worker Node: These nodes execute the tasks and store the actual data (Data Nodes, Node Managers)

Typically, a Hadoop cluster divided into the following layers:

HDFS Layers: The storage layer (with NameNode and DataNodes).

Processing layers: Where MapReduce jobs or YARN resources are executed (Node Manager, Application Master).

Hadoop Architecture Overview

Client Layer: The user or application that interacts with the Hadoop ecosystem.

Resource Management Layer: YARN, which schedules and manages resources.

Storage layer: HDFS for data storage.

Processing layer: MapReduce (or other processing frameworks like spark, Tez) for data processing.

Single Node vs Multi-Node Cluster

↓
All components (NameNode, dataNode, Resource Manager, NodeManager) run on single Machine

↓
Components are distributed across multiple machines for scalability and fault tolerance.

RACK awareness - Hadoop clusters are organised into racks (group of machines)
Data Replication considers rack topology to minimize n/w congestion

Fault Tolerance and Reliability in Hadoop

1. Data Replication : HDFS automatically replicates data blocks across different nodes to ensure fault tolerance. If a DataNode fails, another replica can be used.
2. Job Recovery : In case a node running a MapReduce task fails, YARN or the Resource Manager can reschedule the task to a different node.

Benefits of Hadoop

1. Scalability
2. Fault tolerance
3. Cost - Effective
4. flexible

1.) Scalability — We can scale a Hadoop cluster by adding more machines, which allows for processing large amounts of data.

2.) Fault Tolerance — Data Replication ensure that if a machine fails, the data is still available.

3.) Cost - Effective : — Hadoop works on commodity hardware, making it cheaper to implement than traditional database solutions.

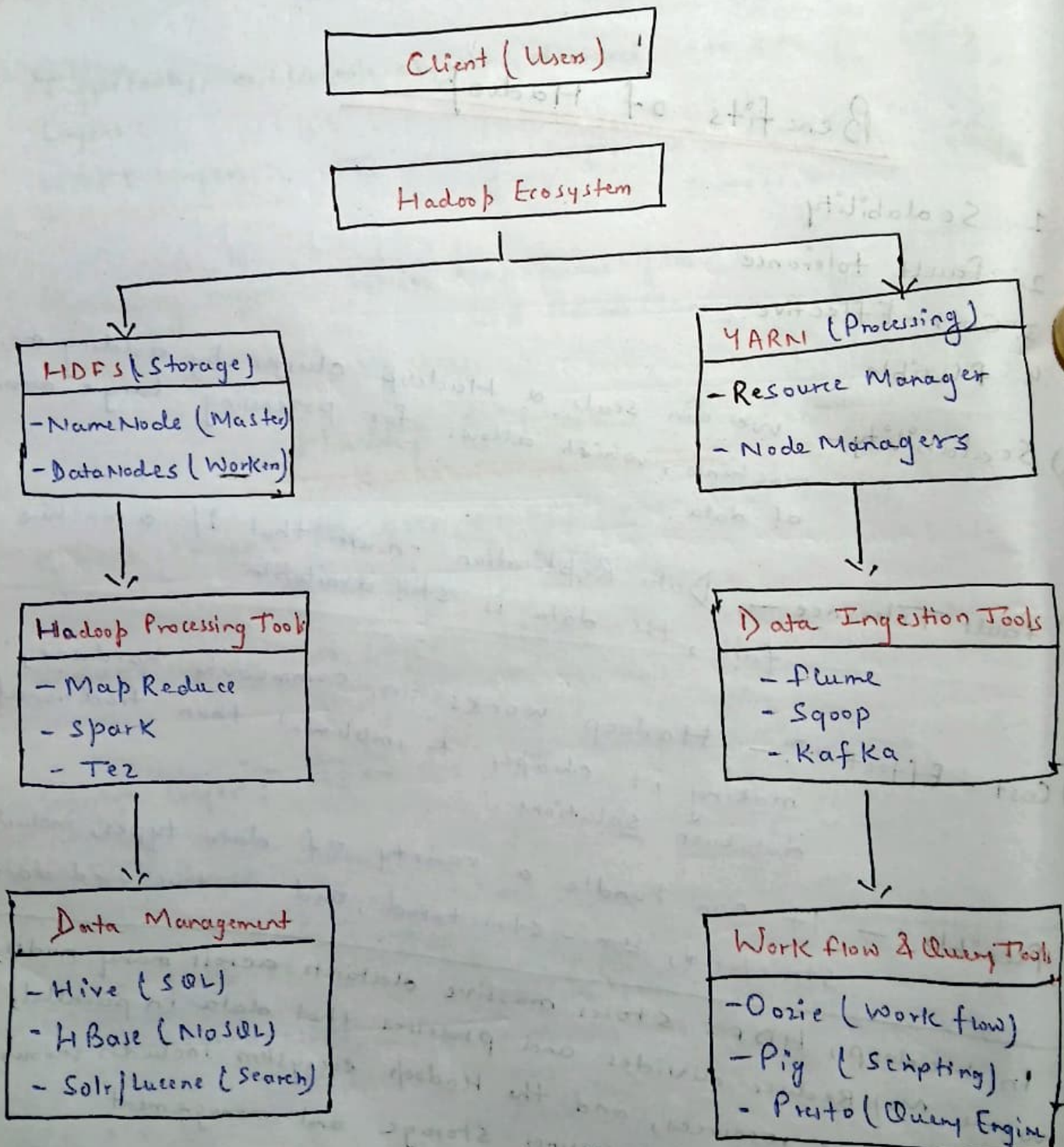
4.) flexible : — It can handle a variety of data types, including structured, semi-structured, and unstructured data.

→ In Hadoop, HDFS stores massive datasets across many nodes, while MapReduce divides and processes that data in parallel, YARN manage resources, and the Hadoop ecosystem includes various projects to help with data processing storage and management.

Hadoop Challenges:

1. Complexity: Require expertise in distributed computing
2. Latency: Not ideal for real-time processing (better handle by Spark)
3. Security: Native Hadoop lacks robust security features (Kerberos can be used)

Hadoop Architecture Diagram



Apache Spark Vs MapReduce (Hadoop)

Spark is best for real-time, fast and iterative processing.

MapReduce is good for batch processing but is slower due to risk I/O.

feature	Apache Spark	Map Reduce (Hadoop)
Processing Model	In-memory computing using RDDs	Disk-based processing (reads & writes to disk)
Speed	up to 100x faster (in memory) and 10x faster (on disk)	Slower due to frequent disk I/O
Ease of Use	Supports high-level APIs in Python, Scala, Java and R	Requires complex Java-based coding
Fault Tolerance	RDDs provides automatic recovery from failure	HDFS replication ensures fault tolerance
Streaming Support	Supports real-time data processing with Spark Streaming	Designed for batch processing only.
Machine Learning	MLlib provides optimized ML capabilities.	Mahout (slower, runs on MapReduce)
Resource Management	Works with YARN, Mesos or standalone cluster managers	Uses YARN for resource allocation.
Best use	Realtime Analytics, ML, interactive queries, graph processing	Batch Processing, ETL, data warehousing.

Apache Spark

Apache Spark is an Open-Source, Distributed computing framework designed for big data processing and analytics.

- Speed, scalability and ease to use.
- Spark enable processing batch and real-time data much faster than Hadoop MapReduce due to its InMemory computation capability.
- 100x faster than Hadoop MapReduce
- Support Multiple Languages - Python, Java, Scala, R
- Works with hadoop, Kubernetes, AWS and more

Spark Architecture

Apache Spark follows a Master-Slave Architecture

Components of Spark Architecture :

1. Driver Program (Master Node) : The main entry point that manages the entire spark application.
2. Cluster Manager — Allocates Resources to Spark Application.
 - Supported cluster managers :—
 - Standalone
 - Apache Mesos
 - Hadoop, YARN
 - Kubernetes
3. Executors (Worker Nodes) — Worker processes that run on multiple nodes and execute tasks in parallel.
4. Tasks — The smallest units of execution in Spark.

5. RDD (Resilient Distributed Dataset) —

The fundamental data structure in Spark for distributed data processing.

Execution flow in Spark

1. User submits an application written in Python, Scala, R, Java
2. Driver Program creates an execution plan and requests resources from the Cluster Manager.
3. Cluster Manager allocates executors (worker nodes).
4. Tasks are distributed to Executors, which process data in parallel.
5. Results are collected and returned to the Driver.

Core Components of Apache Spark

Spark consists of several core components that enables efficient data processing.

1. Resilient Distributed Dataset (RDD) :

RDD is the fundamental data structure in Spark, representing an immutable, distributed collection of objects.

Key features:

1. Fault Tolerant — Automatic recovers lost data
2. Lazy Evaluation — Computation happens only when an action is triggered.
3. Parallel Processing — Data is divided into partitions and processed in parallel.

RDD operations:

1. Transformations

create a new RDD from an existing RDD.
eg - `map()`, `filter()`, `flatMap()`

2. Actions

Trigger computations and return a result.

eg - `collect()`, `count()`, `reduce()`, `saveAsTextFile()`

2. Data frames :

→ A Dataframe is a distributed collection of data organized into columns (similar to a table in SQL). It is based on RDDs but optimized for performance.

advantage of dataframes:

1. Optimized execution using Catalyst Optimizer.
2. Supports various Data format (CSV, JSON, AVro etc)
3. Integrated with SQL queries.

3. Datasets :

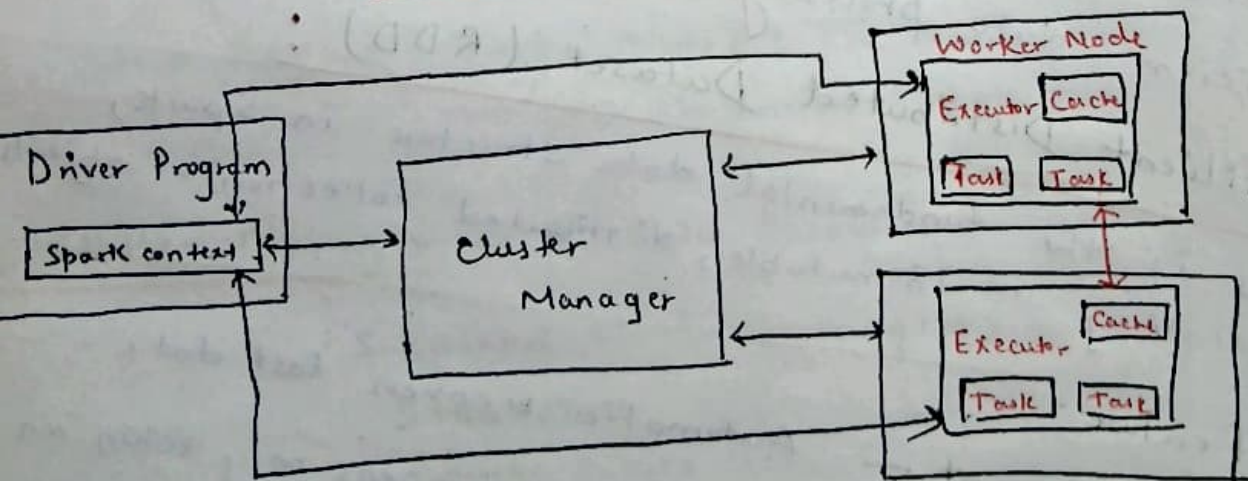
- A dataset is a combination of RDD and Dataframe
- It provides type safety and compile-time checking.

[Faster than RDD]

[Type-safe Operations]

Note : Dataset are available only in Scala & Java

Spark Architecture



Components of a spark cluster.

Spark Core: basic engine for large-scale Parallel & Distributed data processing.

Responsible for —

1. Memory Management
2. Fault Recovery
3. Scheduling, Distributing & Monitoring Jobs
4. Interacting with storage System.

Spark Streaming: used for Processing Real Time data from sources like Kafka, HDFS.
— useful Addition to Core Spark API.
— It enables high-throughput and fault Tolerant stream processing of live data streams.

Spark + Hadoop

Spark + HDFS

Spark + MapReduce

Spark + hadoop YARN

Spark And Hadoop

Spark + Hadoop

↓
Spark is not intended to replace Hadoop but it can be regarded as an extension to it

Spark + MapReduce

↓
MapReduce and Spark are used together where MapReduce is used for batch processing and Spark for real-time processing.

Why Spark?

- Real time processing can be done using Spark but not using Hadoop.
- ML algo are iterative in nature. [earlier tools uses the concept of Disk writing i.e intermediate result is written to disk after each iteration. Making it slow].

Spark Libraries

Apache Spark provides multiple built-in libraries for different types of Processing.

1. Spark SQL
2. Spark Streaming
3. MLib (Machine-Learning Library)
4. Graph X (Graph Processing)

1. Spark SQL : allows querying structured data using SQL-like server syntax.

2. MLib — provides ML algorithm for classifications, regression, clustering etc.

3. Graph X — used for graph analytics & computations.

RHadoop

Why learn R?

The R statistical programming lang is a free open-source package based on the S lang.

- coding style easy. - high performance

R - Operations

R allows performing data analytics by various operations such as:

- Regression
- Classification
- Clustering
- Recommendation

How to link R and Hadoop?

→ RHadoop was developed by Revolution Analytics.

→ RHadoop is available with three main R packages:

1. `rhdfs` - provides HDFS data operations.

2. `rmr` - provides MapReduce execution operations.

3. `rhbase` - input data source at the Hbase.

RHIPE (R and Hadoop Integrated Programming Environment)

- free and open source project.

- widely used for performing Big Data analysis with D & R analysis.

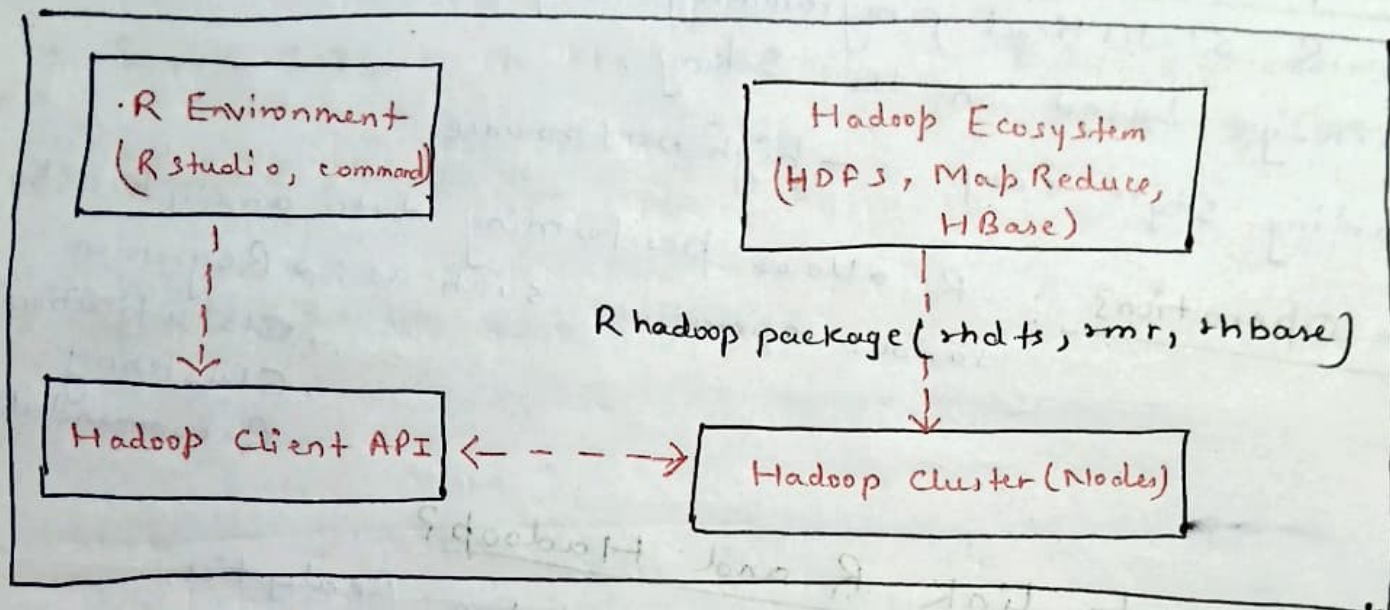
D & R analysis

is used to divide huge data, process it in parallel on a distributed network to produce intermediate output, & finally recombine all this intermediate op into a set.

RHIPE is designed to carry out D & R analysis on complex Big Data in R on the Hadoop platform.

R Hadoop Architecture

R Hadoop follows a client-server model where R interacts with Hadoop components.



How R Hadoop Works:

1. Data is stored in HDFS (Hadoop distributed file system)
2. R code communicates with Hadoop using R Hadoop package
3. Map Reduce jobs are submitted from R to process large data-sets.
4. Results are stored in HDFS or retrieved back into R for analysis.

Advantages of R Hadoop

1. Scalable
2. Powerful Analytics
3. Parallel Processing
4. flexible - Data Handling
5. Open-Source.

Hadoop Streaming is a utility that comes with the Hadoop distribution.

This utility allows us to create and run MapReduce jobs with any executable or script as the Mapper and/or Reducer. This is supported by R, Python, Ruby, so on.

— There is one R Package named Hadoop Streaming that has been developed for performing data analysis on Hadoop clusters with the help of R scripts, which is an interface to Hadoop Streaming with R.

Apache Pig -

- Apache P is an abstraction over MapReduce.
 - AP is designed to handle any kind of data.
 - ~~help~~.
 - All task are encoded in a manner that helps the system to optimize the execution automatically.
- 10 lines of code in Pig == 200 lines of code in Java

Apache Pig composed of 2 components —

1. Pig Latin Programming Language
2. Pig Runtime Environment (in which PL programs are executed.)

- Pig generally used with Hadoop.
- similar to SQL query lang. but applied on a larger dataset and with additional features.
- lang used in Pig is called Pig Latin (similar to SQL)
- It is used to load the data, apply the required filters and dump the data in the required format.
- It require Java Runtime environment to execute the program.

Apache Pig v/s MapReduce

Pig

- Apache Pig is a data flow language.
- easy to perform join operation.
- uses multi query approach, reducing the length of the codes.
- No need of Compilation.
On execution every pig operator is converted internally into a MapReduce job.

Map Reduce

- Map Reduce is a data processing language
- difficult to perform join operation
- requires almost 20 times more the number of lines to perform the same task.
- MR job have a long compilation process.

Pig Latin

- Pig Latin is a high-level data flow language used with Apache Pig, a platform for analyzing large dataset in Hadoop environment.
- It simplifies the creation of MapReduce programs by providing a syntax that abstracts the complexity of Java-based MapReduce coding, making data processing more accessible and efficient.

Pig Latin Data Model:

Pig Latin data model is fully nested and comprises four primary structures:

1. Atom - single data value eg-string / number.
2. Tuple - an ordered set of field. $\uparrow$ relational database
3. Bag - A collection of tuple
4. Map - a set of Key-value pair.

Basic Pig Latin Operation -

1. Load
2. Filter
3. Group
4. Join
5. Order
6. Store.

Architectural Component

1. Parser
2. Optimizer
3. Compiler
4. Execution Engine

1. Parser — Initially Pig scripts are handled by Parser. It checks the syntax of the script, does checking, and miscellaneous checks.

O/p of parser → DAG

2. Optimizer — DAG is passed to the logical optimizer which carries out the logical optimizations such as projection and push down.

3. Compiler — compiles the optimised logical plan into a series of MapReduce jobs.

4. Execution engine — finally the MapReduce jobs are submitted by Hadoop in a sorted order.

finally these MapReduce jobs are executed on Hadoop producing the desired results.